Netflix Content Analysis | Python

Aditya Kushwaha · Python practice project

Analysis of the Netflix titles dataset (6,234 movies and TV shows, added to Netflix between 2008 and January 2020) with Python. The notebook covers data loading, data quality checks, cleaning and feature engineering, and 10 analysis questions on content mix, growth, countries, genres, ratings, duration and content age.

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn

1. Setup and data loading

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 140)
sns.set_theme(style='whitegrid', font_scale=0.95)
PURPLE, NAVY, ORANGE, RED = '#5B2C83', '#1F3A5F', '#E07A1F', '#B20710'

df = pd.read_csv('netflix_titles.csv')
print('Shape:', df.shape)
df.head()
```

```
Shape: (6234, 12)
```

Out[1]:

	show_id	type	title	director	cast	country	date_added	release_y
0	81145628	Movie	Norm of the North: King Sized Adventure	Richard Finn, Tim Maltby	Alan Marriott, Andrew Toth, Brian Dobson, Cole...	United States, India, South Korea, China	September 9, 2019	2
1	80117401	Movie	Jandino: Whatever it Takes	NaN	Jandino Asporaat	United Kingdom	September 9, 2016	2
2	70234439	TV Show	Transformers Prime	NaN	Peter Cullen, Sumalee Montano, Frank Welker, J...	United States	September 8, 2018	2
3	80058654	TV Show	Transformers: Robots in Disguise	NaN	Will Friedle, Darren Criss, Constance Zimmer, ...	United States	September 8, 2018	2
4	80125979	Movie	#realityhigh	Fernando Lebrija	Nesta Cooper, Kate Walsh, John Michael Higgins...	United States	September 8, 2017	2

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6234 entries, 0 to 6233
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       6234 non-null   int64
 1   type          6234 non-null   str
 2   title         6234 non-null   str
 3   director      4265 non-null   str
 4   cast          5664 non-null   str
 5   country       5758 non-null   str
 6   date_added    6223 non-null   str
 7   release_year  6234 non-null   int64
 8   rating        6224 non-null   str
 9   duration      6234 non-null   str
 10  listed_in     6234 non-null   str
 11  description   6234 non-null   str
dtypes: int64(2), str(10)
memory usage: 584.6 KB
```

2. Data quality checks

In [3]:
```
missing = df.isna().sum().to_frame('missing')
missing['missing_pct'] = (missing['missing'] / len(df) * 100).round(1)
missing.sort_values('missing', ascending=False)
```

Out[3]:

	missing	missing_pct
director	1969	31.6
cast	570	9.1
country	476	7.6
date_added	11	0.2
rating	10	0.2
title	0	0.0
show_id	0	0.0
type	0	0.0
release_year	0	0.0
duration	0	0.0
listed_in	0	0.0
description	0	0.0

In [4]:
```
print('Duplicate rows       :', df.duplicated().sum())
print('Duplicate show_id    :', df['show_id'].duplicated().sum())
print('Duplicate title+type :', df.duplicated(subset=['title', 'type',
'release_year']).sum())
```

```
Duplicate rows       : 0
Duplicate show_id    : 0
Duplicate title+type : 3
```

3. Data cleaning and feature engineering

In [5]:
```
clean = df.copy()

# 1. Fill text gaps with an explicit label instead of dropping rows
for col in ['director', 'cast', 'country']:
    clean[col] = clean[col].fillna('Unknown')

# 2. Drop the few rows with no date_added or rating (under 0.4% of rows)
clean = clean.dropna(subset=['date_added', 'rating'])

# 3. Remove exact duplicates on title + type + release year
clean = clean.drop_duplicates(subset=['title', 'type', 'release_year'])

# 4. Parse dates and build time features
clean['date_added'] = pd.to_datetime(clean['date_added'].str.strip(),
format='%B %d, %Y')
clean['year_added'] = clean['date_added'].dt.year
clean['month_added'] = clean['date_added'].dt.month_name()

# 5. Split duration into a number and a unit (minutes for movies, seasons for
TV)
clean['duration_value'] = clean['duration'].str.extract(r'(\d+)').astype(int)
clean['duration_unit'] = np.where(clean['duration'].str.contains('min'),
'min', 'season')

# 6. Main country (first listed) and content age at the time it was added
clean['main_country'] = clean['country'].str.split(',').str[0].str.strip()
clean['content_age'] = clean['year_added'] - clean['release_year']

print('Rows before:', len(df), '| rows after:', len(clean), '| removed:',
len(df) - len(clean))
clean[['title', 'type', 'date_added', 'year_added', 'duration_value',
'duration_unit', 'main_country', 'content_age']].head()
```

```
Rows before: 6234 | rows after: 6211 | removed: 23
```

Out[5]:

	title	type	date_added	year_added	duration_value	duration_unit	main_cou
0	Norm of the North: King Sized Adventure	Movie	2019-09-09	2019	90	min	United St
1	Jandino: Whatever it Takes	Movie	2016-09-09	2016	94	min	Ur King
2	Transformers Prime	TV Show	2018-09-08	2018	1	season	United St
3	Transformers: Robots in Disguise	TV Show	2018-09-08	2018	1	season	United St
4	#realityhigh	Movie	2017-09-08	2017	99	min	United St

In [6]:
```
# Sanity check: negative content age means a data entry error
bad_age = clean[clean['content_age'] < 0]
print('Rows with release year after the date added:', len(bad_age))
clean = clean[clean['content_age'] >= 0]
print('Final clean rows:', len(clean))
```

```
Rows with release year after the date added: 9
Final clean rows: 6202
```

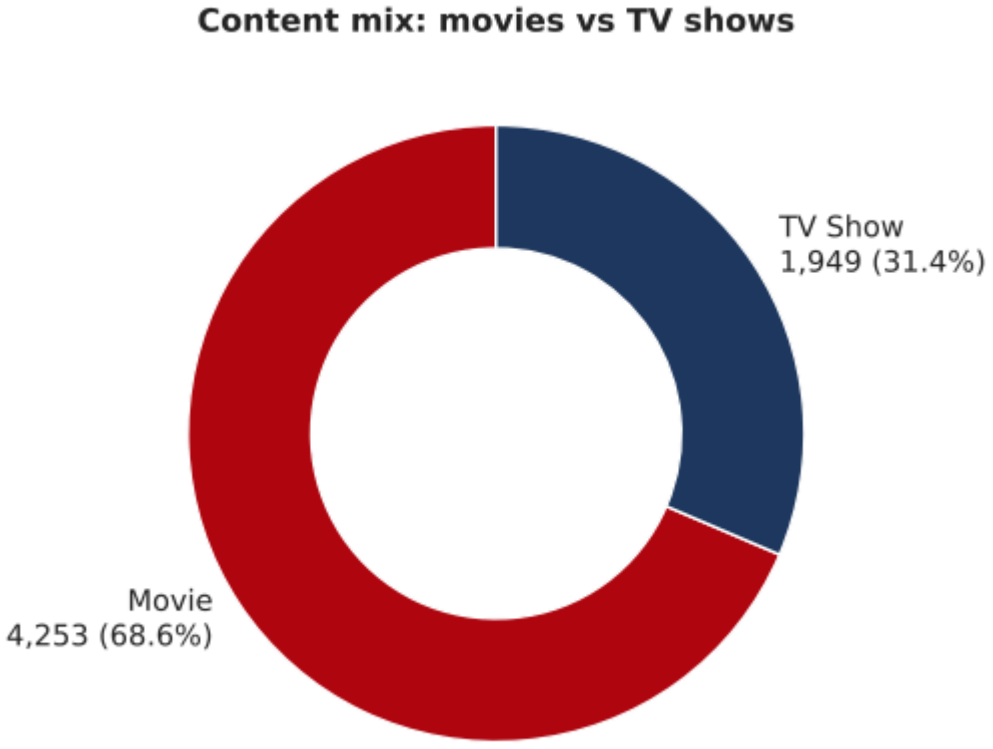
4. Analysis

Q1. Movies vs TV shows

In [7]:
```
mix = clean['type'].value_counts().to_frame('titles')
mix['share_pct'] = (mix['titles'] / mix['titles'].sum() * 100).round(1)
display(mix)

fig, ax = plt.subplots(figsize=(5, 5))
ax.pie(mix['titles'], labels=[f"{i}\n{v:,} ({p}%)" for i, v, p in
zip(mix.index, mix['titles'], mix['share_pct'])],
       colors=[RED, NAVY], startangle=90, wedgeprops=dict(width=0.4))
ax.set_title('Content mix: movies vs TV shows', fontweight='bold')
plt.show()
```

type	titles	share_pct
Movie	4253	68.6
TV Show	1949	31.4



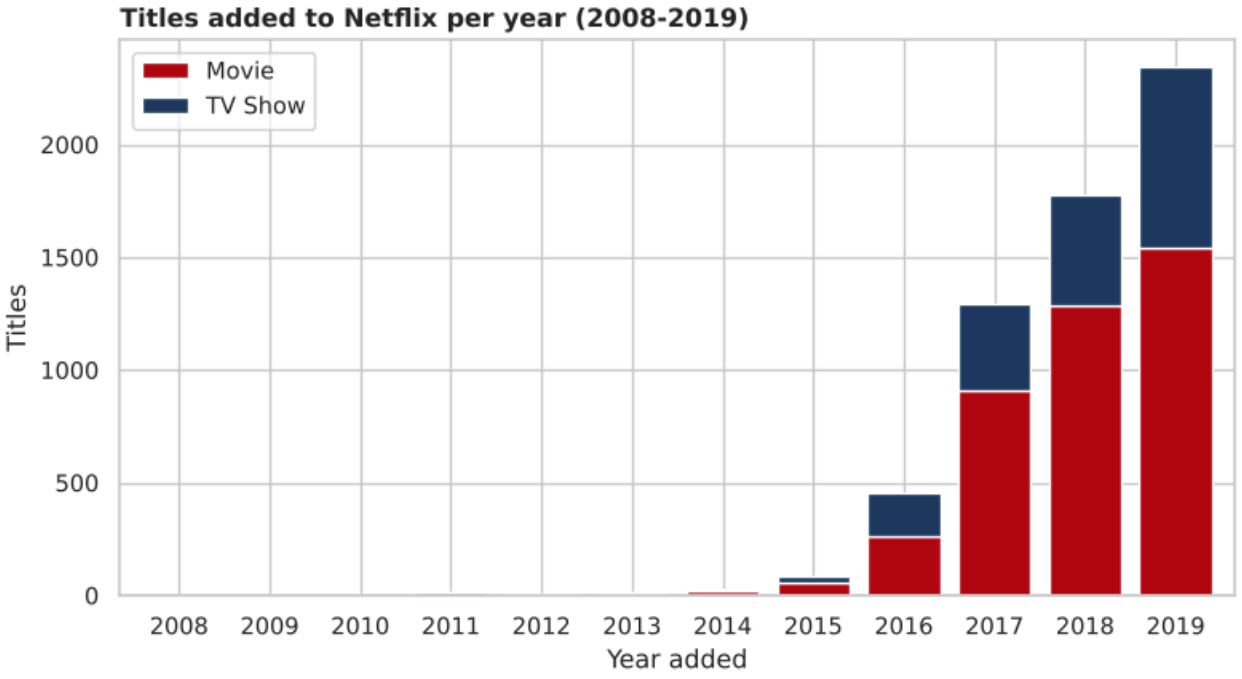
Q2. How fast did the library grow? Titles added per year

In [8]:
```
added = clean[clean['year_added'] <= 2019].groupby(['year_added',
'type']).size().unstack(fill_value=0)
display(added.tail(6))

added.plot(kind='bar', stacked=True, color=[RED, NAVY], figsize=(9, 4.5),
width=0.8)
plt.title('Titles added to Netflix per year (2008-2019)', fontweight='bold',
loc='left')
plt.xlabel('Year added'); plt.ylabel('Titles'); plt.xticks(rotation=0)
plt.legend(title='')
plt.show()

growth = added.sum(axis=1)
print(f"Titles added in 2015: {growth.loc[2015]:,}  |  in 2019:
{growth.loc[2019]:,}  |  growth: {growth.loc[2019] / growth.loc[2015]:.1f}x")
```

type	Movie	TV Show
year_added		
2014	19	6
2015	56	32
2016	262	189
2017	908	387
2018	1289	488
2019	1545	801



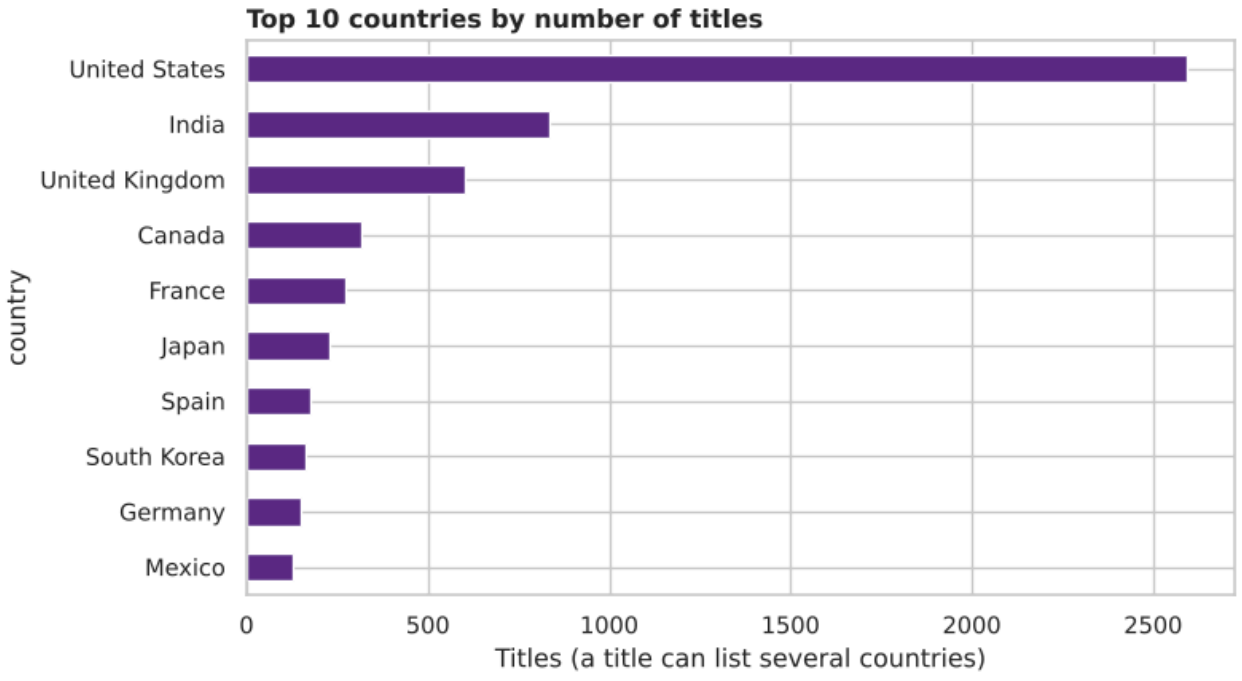

```
Titles added in 2015: 88  |  in 2019: 2,346  |  growth: 26.7x
```

Q3. Top 10 content-producing countries

In [9]:
```
countries = (clean.loc[clean['country'] != 'Unknown', 'country']
             .str.split(',').explode().str.strip())
top_countries = countries.value_counts().head(10)
display(top_countries.to_frame('titles'))

top_countries.sort_values().plot(kind='barh', color=PURPLE, figsize=(8, 4.5))
plt.title('Top 10 countries by number of titles', fontweight='bold',
loc='left')
plt.xlabel('Titles (a title can list several countries)')
plt.show()
```

country	titles
United States	2595
India	837
United Kingdom	601
Canada	317
France	271
Japan	227
Spain	178
South Korea	162
Germany	151
Mexico	128

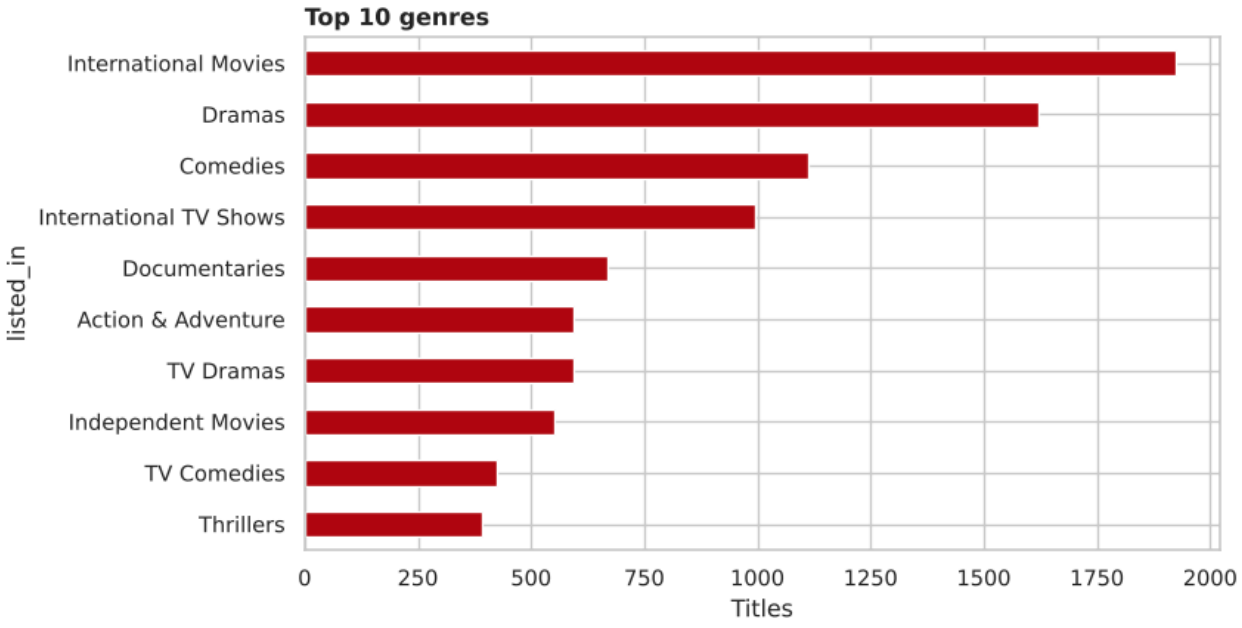


Q4. Most common genres

In [10]:
```
genres = clean['listed_in'].str.split(',').explode().str.strip()
top_genres = genres.value_counts().head(10)
display(top_genres.to_frame('titles'))

top_genres.sort_values().plot(kind='barh', color=RED, figsize=(8, 4.5))
plt.title('Top 10 genres', fontweight='bold', loc='left');
plt.xlabel('Titles')
plt.show()
```

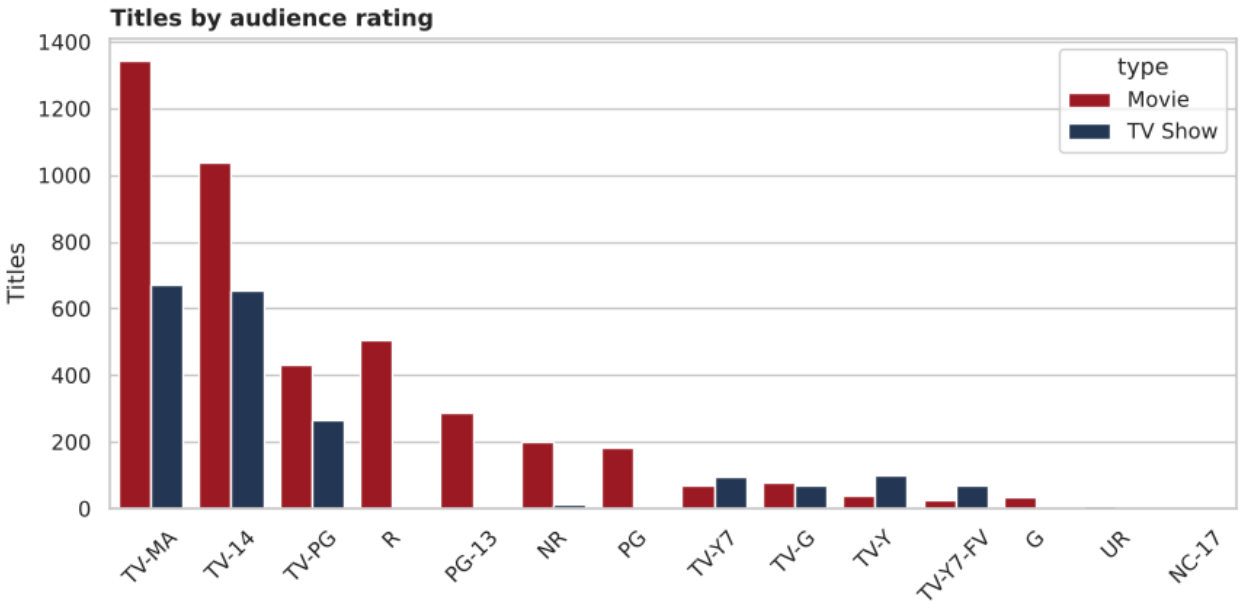
listed_in	titles
International Movies	1925
Dramas	1620
Comedies	1113
International TV Shows	996
Documentaries	668
Action & Adventure	595
TV Dramas	594
Independent Movies	552
TV Comedies	425
Thrillers	392



Q5. Audience ratings: who is the content for?

In [11]:
```
rating_order = clean['rating'].value_counts().index
fig, ax = plt.subplots(figsize=(10, 4.2))
sns.countplot(data=clean, x='rating', hue='type', order=rating_order,
palette=[RED, NAVY], ax=ax)
ax.set_title('Titles by audience rating', fontweight='bold', loc='left');
ax.set_xlabel(''); ax.set_ylabel('Titles')
plt.xticks(rotation=45); plt.show()

mature = clean['rating'].isin(['TV-MA', 'R', 'NC-17']).mean() * 100
print(f'Share of titles rated for mature audiences (TV-MA, R, NC-17):
{mature:.1f}%')
```



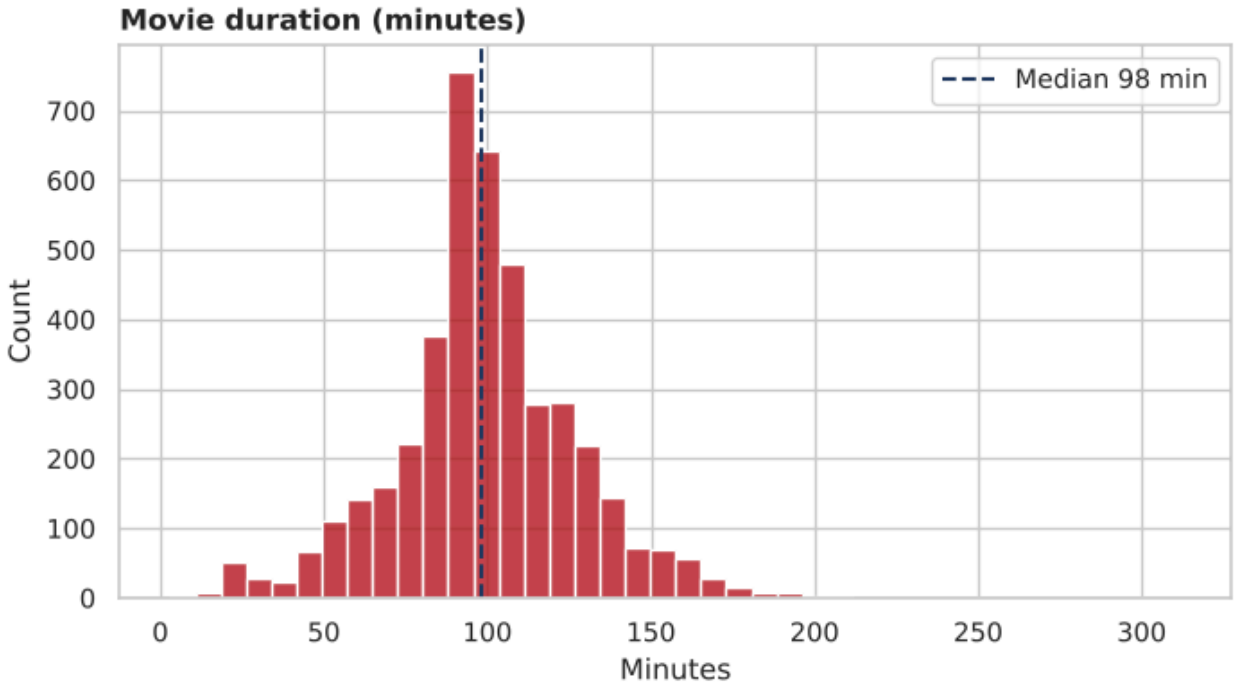

```
Share of titles rated for mature audiences (TV-MA, R, NC-17): 40.8%
```

Q6. Movie length distribution

In [12]:
```
movies = clean[clean['type'] == 'Movie']
print(movies['duration_value'].describe().round(1))

fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(movies['duration_value'], bins=40, color=RED, ax=ax)
ax.axvline(movies['duration_value'].median(), color=NAVY, ls='--',
label=f"Median {movies['duration_value'].median():.0f} min")
ax.set_title('Movie duration (minutes)', fontweight='bold', loc='left');
ax.set_xlabel('Minutes'); ax.legend()
plt.show()
```

```
count    4253.0
mean       99.1
std        28.1
min         3.0
25%        86.0
50%        98.0
75%       115.0
max       312.0
Name: duration_value, dtype: float64
```



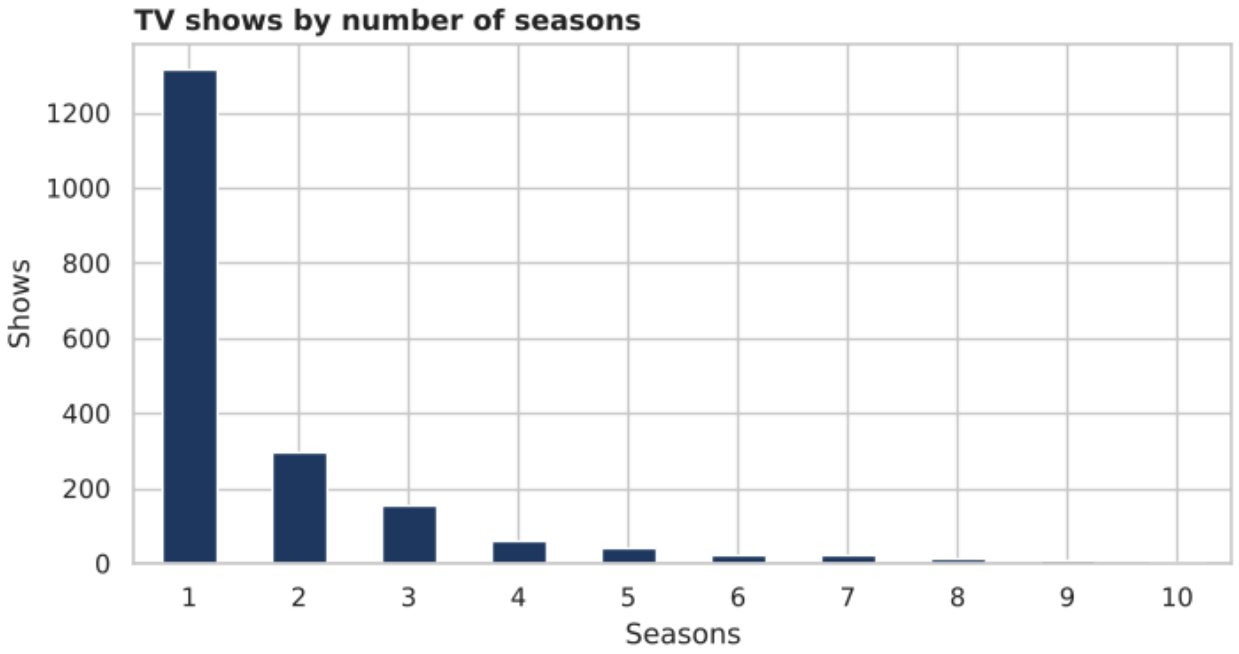
Q7. TV shows: how many seasons?

In [13]:
```
tv = clean[clean['type'] == 'TV Show']
seasons = tv['duration_value'].value_counts().sort_index()
display(seasons.head(8).to_frame('shows'))
print(f"Shows with only 1 season: {(tv['duration_value'] == 1).mean() *
100:.1f}%")

seasons.head(10).plot(kind='bar', color=NAVY, figsize=(8, 3.8))
plt.title('TV shows by number of seasons', fontweight='bold', loc='left');
plt.xlabel('Seasons'); plt.ylabel('Shows'); plt.xticks(rotation=0)
plt.show()
```

duration_value	shows
1	1315
2	298
3	157
4	59
5	44
6	22
7	21
8	16

```
Shows with only 1 season: 67.5%
```

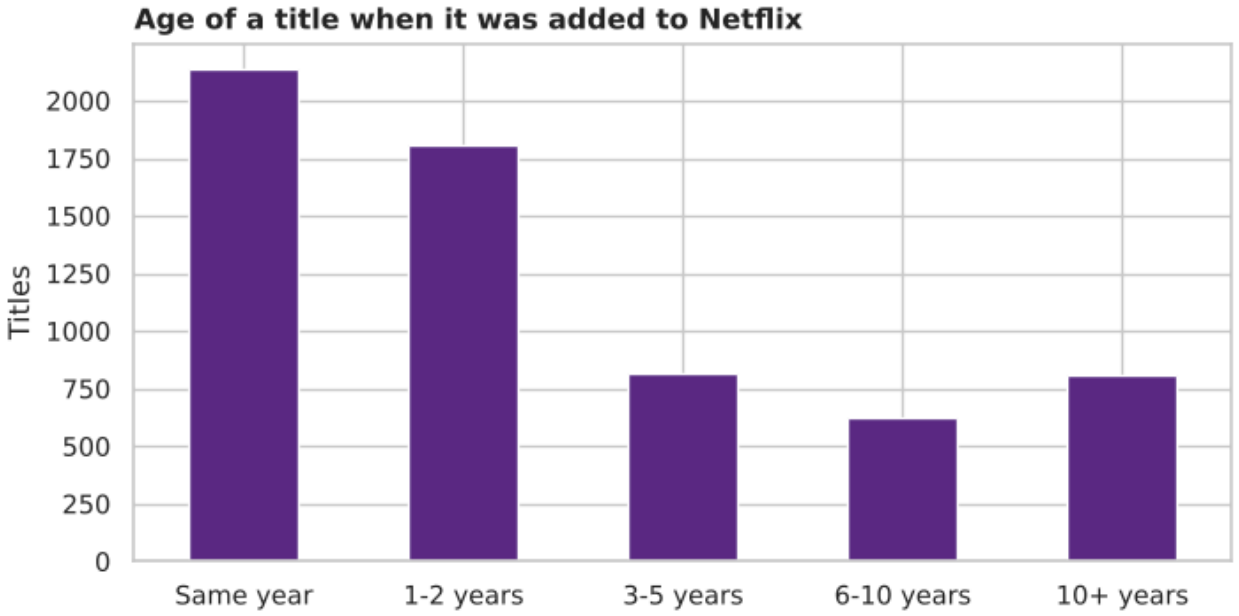


Q8. Content age: new releases or older catalogue?

In [14]:
```
age_bins = pd.cut(clean['content_age'], bins=[-1, 0, 2, 5, 10, 100],
                  labels=['Same year', '1-2 years', '3-5 years', '6-10
years', '10+ years'])
age = age_bins.value_counts().reindex(['Same year', '1-2 years', '3-5 years',
'6-10 years', '10+ years'])
display((age / age.sum() * 100).round(1).to_frame('share_pct'))

age.plot(kind='bar', color=PURPLE, figsize=(8, 3.8))
plt.title('Age of a title when it was added to Netflix', fontweight='bold',
loc='left'); plt.xlabel(''); plt.ylabel('Titles'); plt.xticks(rotation=0)
plt.show()
```

content_age	share_pct
Same year	34.5
1-2 years	29.1
3-5 years	13.2
6-10 years	10.1
10+ years	13.1



Q9. Top directors (excluding unknown)

In [15]:
```
directors = clean.loc[clean['director'] != 'Unknown',
'director'].str.split(',').explode().str.strip()
directors.value_counts().head(10).to_frame('titles')
```

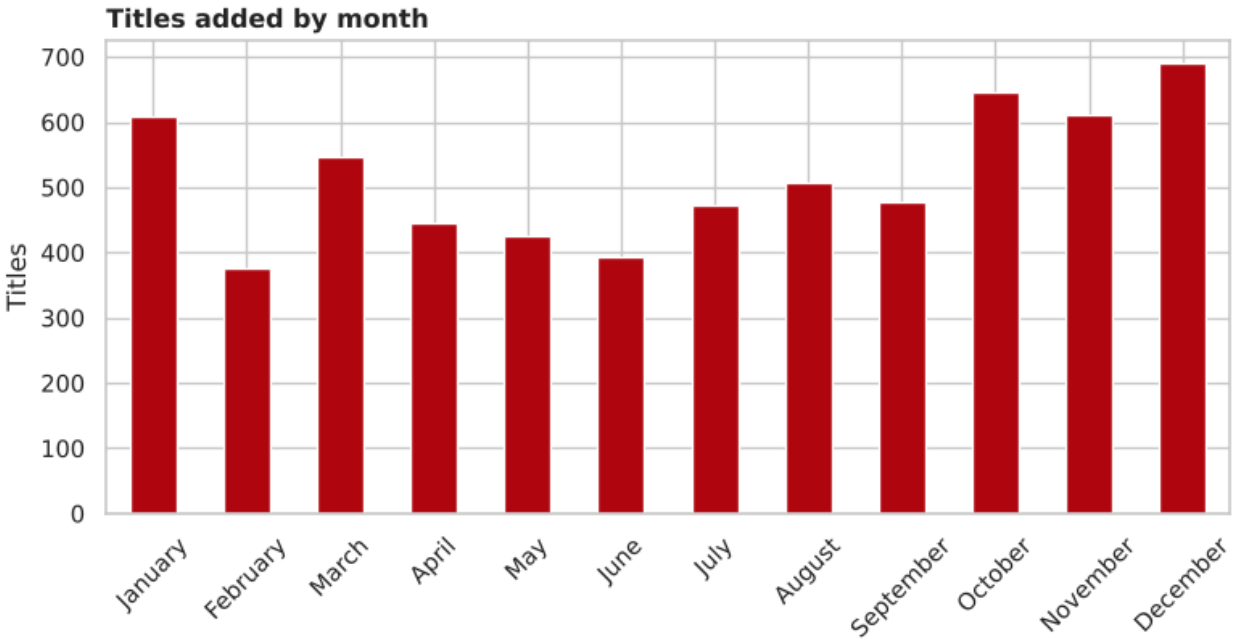
Out[15]:

director	titles
Jan Suter	21
Raúl Campos	19
Jay Karas	14
Marcus Raboy	14
Jay Chapman	12
Martin Scorsese	9
Steven Spielberg	9
Umesh Mehra	8
Shannon Hartman	8
Lance Bangs	8

Q10. Which month does Netflix add the most content?

In [16]:
```
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
'August', 'September', 'October', 'November', 'December']
by_month = clean['month_added'].value_counts().reindex(months)

by_month.plot(kind='bar', color=RED, figsize=(9, 3.8))
plt.title('Titles added by month', fontweight='bold', loc='left');
plt.xlabel(''); plt.ylabel('Titles'); plt.xticks(rotation=45)
plt.show()
print('Busiest month:', by_month.idxmax(), f'({by_month.max():,} titles)', '|
quietest:', by_month.idxmin(), f'({by_month.min():,} titles)')
```




```
Busiest month: December (690 titles) | quietest: February (377 titles)
```

5. Key findings

In [17]:
```
summary = {
    'Titles after cleaning': f"{len(clean):,}",
    'Movies share': f"{(clean['type'] == 'Movie').mean() * 100:.1f}%",
    'Top country': top_countries.index[0],
    'Top genre': top_genres.index[0],
    'Most common rating': clean['rating'].mode()[0],
    'Median movie length': f"{movies['duration_value'].median():.0f} min",
    'TV shows with 1 season': f"{(tv['duration_value'] == 1).mean() *
100:.1f}%",
    'Busiest month for new titles': by_month.idxmax(),
}
pd.Series(summary).to_frame('value')
```

Out[17]:

	value
Titles after cleaning	6,202
Movies share	68.6%
Top country	United States
Top genre	International Movies
Most common rating	TV-MA
Median movie length	98 min
TV shows with 1 season	67.5%
Busiest month for new titles	December

In [18]:
```
clean.to_csv('netflix_titles_clean.csv', index=False)
print('Saved netflix_titles_clean.csv')
```

```
Saved netflix_titles_clean.csv
```